



DOCUMENTO 11

Documentación Final del Proyecto

Documento consolidado del sistema de gestión de parqueadero

Proyecto	PARQUEADERO — Sistema de gestión de parqueadero
Institución	I.E.R. Santa María
Integrantes	Loren
Docente	Liseth Giraldo
Versión / Fecha	1.0 — 1 de octubre de 2026

Contenido

1. Presentación
2. Descripción general, problemática y objetivos
3. Requisitos funcionales
4. Requisitos no funcionales
5. Historias de usuario
6. Arquitectura del sistema
7. Modelo entidad-relación
8. Base de datos MySQL
9. Funcionamiento del sistema
10. Tecnologías y documentación técnica
11. Estado del proyecto
12. Conclusiones

1. Presentación

Alcance de este documento y fuentes de información

Este documento distingue tres tipos de información:

- **EXISTENTE** Elementos verificados en el repositorio del proyecto a la fecha de elaboración.
- **PROPUESTO** Diseño planteado para el sistema de parqueadero, aún no implementado en el código.
- **PENDIENTE** Datos que deben ser completados por el equipo del proyecto.

El video de referencia corresponde a una sesión de clase que muestra la **metodología y las herramientas** de desarrollo (repositorio en GitHub, frontend PWA con Vite, base de datos MySQL en Clever Cloud y despliegue en Vercel). El video **no muestra funcionalidades del sistema de parqueadero**; por esta razón, las funcionalidades descritas se presentan como diseño propuesto.

1.1 Ficha del proyecto

Campo	Información
Nombre del proyecto	PARQUEADERO
Marca	Parqueadero — Systems & Programación
Repositorio	github.com/camilotobon34-sudo/Parkinloock-
Nombre técnico en el código	Parkinlock (repositorio <i>Parkinloock-</i> , carpetas <i>frontend-Parkinlock-</i> y <i>backend-Parkinlock</i>)
Tipo de aplicación	Aplicación web progresiva (PWA)
Institución	I.E.R. Santa María
Integrantes	Loren
Docente	Liseth Giraldo
Versión del documento	1.0
Fecha	1 de octubre de 2026

2. Descripción general, problemática y objetivos

2.1 Nombre del proyecto

PARQUEADERO — Systems & Programación. En el repositorio el proyecto se identifica con el nombre técnico **Parkinlock**.

2.2 Descripción

PARQUEADERO es un sistema web para la gestión del parqueadero de una institución. Según la descripción registrada en el repositorio del proyecto, el parqueadero es un *"espacio destinado al estacionamiento organizado y seguro de vehículos de estudiantes, profesores y personal de la institución, facilitando el control y uso adecuado de los espacios disponibles"*.

El sistema se plantea como una aplicación web progresiva (PWA) que permite registrar los vehículos autorizados de la comunidad institucional, controlar sus ingresos y salidas, conocer en tiempo real la disponibilidad de espacios y consultar el historial de uso del parqueadero.

2.3 Problema que busca solucionar

El control del parqueadero institucional se realiza normalmente de forma manual (planillas en papel o registro visual del personal de vigilancia), lo que genera:

- Dificultad para saber cuántos espacios están disponibles en un momento dado.
- Falta de un registro confiable de qué vehículo ingresó, a qué hora y en qué espacio se ubicó.
- Imposibilidad de verificar rápidamente si un vehículo pertenece a un miembro de la institución.
- Pérdida o deterioro de la información histórica y ausencia de reportes de ocupación.

Dato pendiente

Si el equipo cuenta con datos reales del parqueadero (número de espacios, cantidad de vehículos diarios, forma actual de control), deben incluirse aquí para sustentar la problemática. **[Pendiente de completar]**

2.4 Justificación

Digitalizar el control del parqueadero permite organizar el uso de los espacios, mejorar la seguridad de los vehículos de la comunidad institucional y contar con información confiable para la toma de decisiones. Al construirse como PWA, el sistema puede usarse desde el navegador del computador de portería o desde un teléfono móvil, sin necesidad de instalar software adicional.

2.5 Objetivo general

Desarrollar un sistema web progresivo que permita gestionar el ingreso, la salida y la ocupación de los espacios del parqueadero institucional para los vehículos de estudiantes, profesores y personal de la institución.

2.6 Objetivos específicos

1. Registrar los usuarios de la institución y los vehículos autorizados asociados a cada uno.
2. Controlar el ingreso y la salida de vehículos, asignando un espacio disponible.
3. Mostrar la disponibilidad de espacios del parqueadero por zona y tipo de vehículo.
4. Almacenar el historial de ingresos y salidas en una base de datos MySQL.
5. Generar consultas y reportes de ocupación para la administración.
6. Restringir el acceso a las funciones según el rol del usuario (administrador, vigilante, usuario).

2.7 Alcance

El sistema contempla la gestión de usuarios, vehículos, zonas, espacios y registros de ingreso/salida del parqueadero de una sola institución.

Incluido en el alcance (propuesto)	Fuera del alcance
Inicio y cierre de sesión con roles	Cobro de tarifas o pagos en línea
Registro de usuarios y vehículos	Lectura automática de placas por cámara
Registro de ingreso y salida de vehículos	Control de barreras o hardware físico
Consulta de disponibilidad de espacios	Gestión de varias sedes o parqueaderos
Historial y reportes de ocupación	Reservas anticipadas de espacios

Los elementos fuera del alcance podrán evaluarse en versiones futuras.

2.8 Usuarios del sistema

Actor	Descripción
Administrador	Gestiona usuarios, vehículos, zonas y espacios; consulta reportes de ocupación.
Vigilante	Personal de portería que registra los ingresos y salidas de vehículos y consulta la disponibilidad.
Usuario	Estudiante, profesor o integrante del personal de la institución que tiene vehículos registrados y consulta su historial y la disponibilidad.

2.9 Beneficios esperados

- Conocimiento en tiempo real de los espacios disponibles.
- Mayor seguridad: solo ingresan vehículos registrados y queda constancia de cada movimiento.
- Agilidad en la portería al buscar vehículos por placa.
- Historial confiable y reportes para la administración.
- Acceso desde computador o móvil gracias a la tecnología PWA.

3. Requisitos funcionales

Los siguientes requisitos funcionales describen el comportamiento **propuesto** del sistema de parqueadero. A la fecha, ninguno de ellos está implementado en el código del repositorio.

3.1 Actores

Actor	Rol en el sistema
Administrador	Configura el parqueadero y gestiona usuarios y vehículos.
Vigilante	Opera la portería: ingresos, salidas y consultas.
Usuario	Miembro de la institución con vehículos registrados.
Sistema	Validaciones automáticas realizadas por la aplicación.

3.2 Tabla de requisitos funcionales

Código	Requisito funcional	Descripción	Actor	Prioridad
RF-01	Iniciar sesión	El sistema debe permitir el ingreso mediante correo institucional y contraseña, validando las credenciales contra la base de datos.	Administrador, Vigilante, Usuario	Alta
RF-02	Cerrar sesión	El sistema debe permitir finalizar la sesión activa de forma segura.	Administrador, Vigilante, Usuario	Alta
RF-03	Control de acceso por rol	El sistema debe mostrar únicamente las opciones permitidas para el rol del usuario autenticado.	Sistema	Alta
RF-04	Gestionar usuarios	El administrador debe poder registrar, consultar, editar y desactivar usuarios (estudiantes, profesores y personal).	Administrador	Alta
RF-05	Gestionar vehículos	El sistema debe permitir registrar, editar y desactivar los vehículos asociados a un usuario (placa, tipo, marca, modelo y color).	Administrador	Alta
RF-06	Validar placa única	El sistema no debe permitir registrar dos vehículos con la misma placa.	Sistema	Alta
RF-07	Gestionar zonas	El administrador debe poder crear y editar las zonas del parqueadero.	Administrador	Media

Código	Requisito funcional	Descripción	Actor	Prioridad
RF-08	Gestionar espacios	El administrador debe poder crear espacios, asignarlos a una zona y tipo de vehículo, y cambiar su estado (disponible, ocupado, inhabilitado).	Administrador	Alta
RF-09	Registrar ingreso de vehículo	El vigilante debe poder registrar el ingreso de un vehículo buscándolo por placa y asignándole un espacio disponible del tipo correspondiente.	Vigilante	Alta
RF-10	Validar vehículo autorizado	Al registrar un ingreso, el sistema debe verificar que el vehículo esté registrado, activo y que no tenga un ingreso abierto.	Sistema	Alta
RF-11	Registrar salida de vehículo	El vigilante debe poder registrar la salida de un vehículo; el sistema guarda la fecha y hora y libera el espacio.	Vigilante	Alta
RF-12	Consultar disponibilidad	El sistema debe mostrar la cantidad de espacios disponibles y ocupados por zona y tipo de vehículo.	Administrador, Vigilante, Usuario	Alta
RF-13	Consultar vehículos en el parqueadero	El sistema debe listar los vehículos que se encuentran actualmente dentro del parqueadero.	Administrador, Vigilante	Media
RF-14	Consultar historial	El sistema debe permitir consultar el historial de ingresos y salidas filtrando por placa y rango de fechas. El usuario solo ve el historial de sus propios vehículos.	Administrador, Vigilante, Usuario	Media
RF-15	Generar reporte de ocupación	El administrador debe poder consultar un reporte con el número de ingresos por día y la ocupación por zona.	Administrador	Baja
RF-16	Consultar mis vehículos	El usuario debe poder ver los vehículos que tiene registrados a su nombre.	Usuario	Media

3.3 Criterio de prioridad

Prioridad	Significado
Alta	Indispensable para la primera versión funcional del sistema.
Media	Importante; puede entregarse en una segunda iteración.
Baja	Deseable; mejora el sistema pero no bloquea su uso.

4. Requisitos no funcionales

Se presentan los requisitos no funcionales del sistema indicando, para cada uno, si está **respaldado por la configuración existente** del proyecto o si es una meta **propuesta**.

Código	Categoría	Requisito	Estado	Respaldo
RNF-01	Usabilidad	La interfaz debe ser adaptable (responsive) para usarse en computador y en teléfono móvil.	PROPUESTO	La plantilla actual incluye <i>viewport</i> móvil en <i>index.html</i> .
RNF-02	Usabilidad	Las acciones principales (ingreso y salida) deben completarse en máximo tres pasos desde el panel del vigilante.	PROPUESTO	—
RNF-03	Usabilidad	La aplicación debe poder instalarse en el dispositivo como PWA, con nombre e íconos propios.	EXISTENTE	vite-plugin-pwa configurado con manifest "Parkinlock" e íconos de 64, 192 y 512 px y maskable.
RNF-04	Seguridad	Las contraseñas deben almacenarse cifradas con un algoritmo de hash (por ejemplo, bcrypt), nunca en texto plano.	PROPUESTO	—
RNF-05	Seguridad	Las funciones deben protegerse según el rol del usuario autenticado.	PROPUESTO	—
RNF-06	Seguridad	Las credenciales de la base de datos deben manejarse en variables de entorno y no subirse al repositorio.	PROPUESTO	El <i>.gitignore</i> del repositorio ya excluye archivos <i>.env</i> .
RNF-07	Rendimiento	Los recursos estáticos del frontend deben almacenarse en caché mediante service worker para acelerar las cargas posteriores.	EXISTENTE	El build genera <i>sw.js</i> con Workbox (modo generateSW, actualización automática).
RNF-08	Rendimiento	Las consultas de disponibilidad deben responder en menos de 2 segundos en condiciones normales.	PROPUESTO	—
RNF-09	Disponibilidad	El frontend debe desplegarse en un servicio de hosting en la nube (Vercel, según la metodología del curso).	PROPUESTO	Metodología mostrada en el video de referencia.
RNF-10	Disponibilidad	La base de datos MySQL debe alojarse en un servicio administrado en la nube (Clever Cloud, según la metodología del curso).	PROPUESTO	Metodología mostrada en el video de referencia.

Código	Categoría	Requisito	Estado	Respaldo
RNF-11	Compatibilidad	La aplicación debe funcionar en las versiones actuales de navegadores Chrome, Edge y Firefox.	PROPUESTO	—
RNF-12	Mantenibilidad	El código fuente debe versionarse en GitHub, separando frontend y backend en carpetas independientes.	EXISTENTE	Repositorio con carpetas frontend-Parkinlock- y backend-Parkinlock.
RNF-13	Mantenibilidad	El código del frontend debe analizarse con un linter.	EXISTENTE	oxlint configurado (script npm run lint).
RNF-14	Escalabilidad	El modelo de datos debe permitir agregar nuevas zonas, espacios y tipos de vehículo sin modificar la estructura de las tablas.	PROPUESTO	Ver modelo entidad-relación propuesto.

5. Historias de usuario

Las historias de usuario describen, desde la perspectiva de cada actor, las funcionalidades **propuestas** del sistema de parqueadero. Cada historia está relacionada con los requisitos funcionales del documento 02.

Código	Historia de usuario	Criterios de aceptación	Prioridad
HU-01 RF-01, RF-03	Como usuario del sistema, quiero iniciar sesión con mi correo y contraseña, para acceder a las funciones que corresponden a mi rol.	• Si las credenciales son correctas, el sistema muestra el panel según el rol. • Si son incorrectas, se muestra el mensaje "Correo o contraseña incorrectos". • Un usuario inactivo no puede iniciar sesión.	Alta
HU-02 RF-02	Como usuario del sistema, quiero cerrar mi sesión, para evitar que otra persona use mi cuenta en el mismo dispositivo.	• Al cerrar sesión se elimina la sesión activa. • El sistema redirige a la pantalla de inicio de sesión.	Alta
HU-03 RF-04	Como administrador, quiero registrar a los estudiantes, profesores y personal, para que puedan tener vehículos autorizados en el parqueadero.	• El formulario solicita documento, nombres, apellidos, correo, teléfono, tipo de usuario y rol. • No se permite registrar un documento o correo ya existente. • El usuario queda en estado ACTIVO al crearse.	Alta
HU-04 RF-05, RF-06	Como administrador, quiero registrar los vehículos de cada usuario, para controlar qué vehículos están autorizados para ingresar.	• Se solicita placa, tipo de vehículo, marca, modelo y color. • Si la placa ya existe, se muestra el mensaje "La placa ya está registrada". • El vehículo queda asociado al usuario seleccionado.	Alta
HU-05 RF-07, RF-08	Como administrador, quiero crear zonas y espacios del parqueadero, para reflejar la distribución real del parqueadero en el sistema.	• Cada espacio tiene un código único, una zona y un tipo de vehículo. • Un espacio nuevo queda en estado DISPONIBLE. • Se puede inhabilitar un espacio por mantenimiento.	Alta

Código	Historia de usuario	Criterios de aceptación	Prioridad
HU-06 RF-09, RF-10	Como vigilante, quiero registrar el ingreso de un vehículo por su placa, para llevar el control de quién entra al parqueadero.	- Al digitar la placa, el sistema muestra el propietario y el tipo de vehículo. - Si la placa no está registrada o el vehículo está inactivo, se muestra "Vehículo no autorizado" y no se permite el ingreso. - Si el vehículo ya tiene un ingreso abierto, se muestra "El vehículo ya se encuentra en el parqueadero". - Al confirmar, se guarda la fecha y hora y el espacio pasa a OCUPADO.	Alta
HU-07 RF-11	Como vigilante, quiero registrar la salida de un vehículo, para liberar el espacio y dejar constancia de la hora de salida.	- Solo se puede registrar la salida de vehículos con un ingreso abierto. - Se guarda la fecha y hora de salida y el vigilante que la registró. - El espacio vuelve a estado DISPONIBLE.	Alta
HU-08 RF-12	Como vigilante, quiero ver la disponibilidad de espacios por zona y tipo de vehículo, para indicar a los conductores dónde pueden estacionar.	- Se muestra el total de espacios disponibles, ocupados e inhabilitados por zona. - La información se actualiza después de cada ingreso o salida.	Alta
HU-09 RF-12	Como usuario, quiero consultar si hay espacios disponibles, para saber antes de llegar si podré estacionar.	Se muestra la cantidad de espacios disponibles por tipo de vehículo.	Media
HU-10 RF-14, RF-16	Como usuario, quiero consultar mis vehículos y su historial de ingresos, para llevar control del uso que hago del parqueadero.	- El usuario solo ve sus propios vehículos e historial. - El historial muestra placa, espacio, fecha/hora de ingreso y de salida.	Media
HU-11 RF-13	Como vigilante, quiero ver la lista de vehículos que están dentro del parqueadero, para verificar rápidamente la ocupación actual.	Se listan los registros sin fecha de salida con placa, propietario, espacio y hora de ingreso.	Media

Código	Historia de usuario	Criterios de aceptación	Prioridad
HU-12 RF-15	Como administrador, quiero consultar un reporte de ocupación por fechas, para tomar decisiones sobre el uso del parqueadero.	• Se puede filtrar por rango de fechas. • El reporte muestra ingresos por día y ocupación por zona.	Baja

6. Arquitectura del sistema

El sistema PARQUEADERO sigue una arquitectura cliente-servidor de tres capas: **frontend** (lo que ve el usuario), **backend** (la lógica y las reglas del negocio) y **base de datos** (donde se guarda la información).

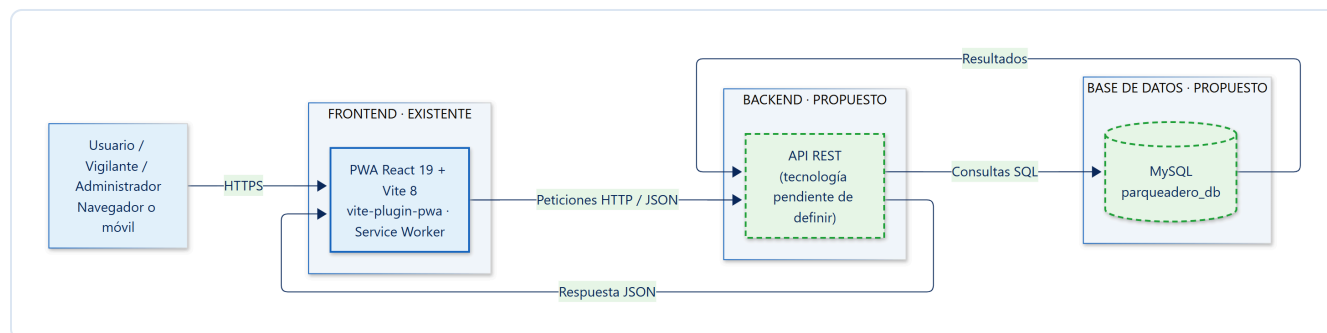


Figura 1. Arquitectura del sistema. Línea continua: componente existente. Línea punteada: componente propuesto.

6.1 Frontend EXISTENTE

Ubicado en la carpeta `frontend-Parkinlock`. Es una aplicación web progresiva construida con:

Tecnología	Versión	Uso
React	19.2	Construcción de la interfaz mediante componentes.
Vite	8.3	Servidor de desarrollo y empaquetado para producción.
vite-plugin-pwa	1.3	Genera el manifest y el service worker (Workbox) para que la app sea instalable y funcione con caché.
@vite-pwa/assets-generator	1.0.4	Genera los íconos de la PWA a partir de una imagen base.
oxlint	1.81	Análisis estático del código (linter).

Estado actual

El frontend contiene la estructura base generada por Vite y la configuración PWA. Las pantallas del parqueadero (inicio de sesión, panel, ingresos, salidas, etc.) aún no están desarrolladas.

6.2 Backend PROPUESTO

Ubicado en la carpeta `backend-Parkinlock`, que actualmente está **vacía**. Su función será exponer una API REST que reciba las peticiones del frontend, aplique las reglas del negocio (validar placa, asignar espacios, controlar roles) y consulte la base de datos.

Tecnología del backend: [Pendiente de definir por el equipo]

6.3 Base de datos PROPUESTO

Base de datos relacional **MySQL** (`parqueadero_db`) con 7 tablas: roles, usuarios, tipos_vehiculo, vehiculos, zonas, espacios y registros. Según la metodología del video de referencia, se alojaría en **Clever Cloud**.

6.4 Comunicación entre componentes

1. El navegador carga la PWA desde el servidor de hosting (en la metodología del curso, Vercel).
2. El frontend envía peticiones HTTP con datos en formato JSON a la API del backend.
3. El backend valida la petición y ejecuta consultas SQL sobre MySQL.
4. MySQL devuelve los resultados al backend.
5. El backend responde al frontend en JSON y la interfaz se actualiza.

6.5 Flujo de información: ejemplo "registrar ingreso"

Paso	Componente	Acción
1	Frontend	El vigilante digita la placa y pulsa "Buscar".
2	Backend	Consulta el vehículo, verifica que esté activo y que no tenga un ingreso abierto.
3	Base de datos	Devuelve los datos del vehículo y los espacios disponibles de su tipo.
4	Frontend	Muestra el propietario y la lista de espacios; el vigilante confirma.
5	Backend + BD	Inserta el registro y marca el espacio como OCUPADO en una transacción.
6	Frontend	Muestra el mensaje "Ingreso registrado correctamente".

7. Modelo entidad-relación

Modelo propuesto

El repositorio **no contiene actualmente una base de datos** (no existen archivos SQL, migraciones ni conexión a MySQL). Por lo tanto, el siguiente modelo es un **diseño propuesto** derivado de los requisitos funcionales, y deberá actualizarse si la implementación final cambia.

7.1 Diagrama entidad-relación

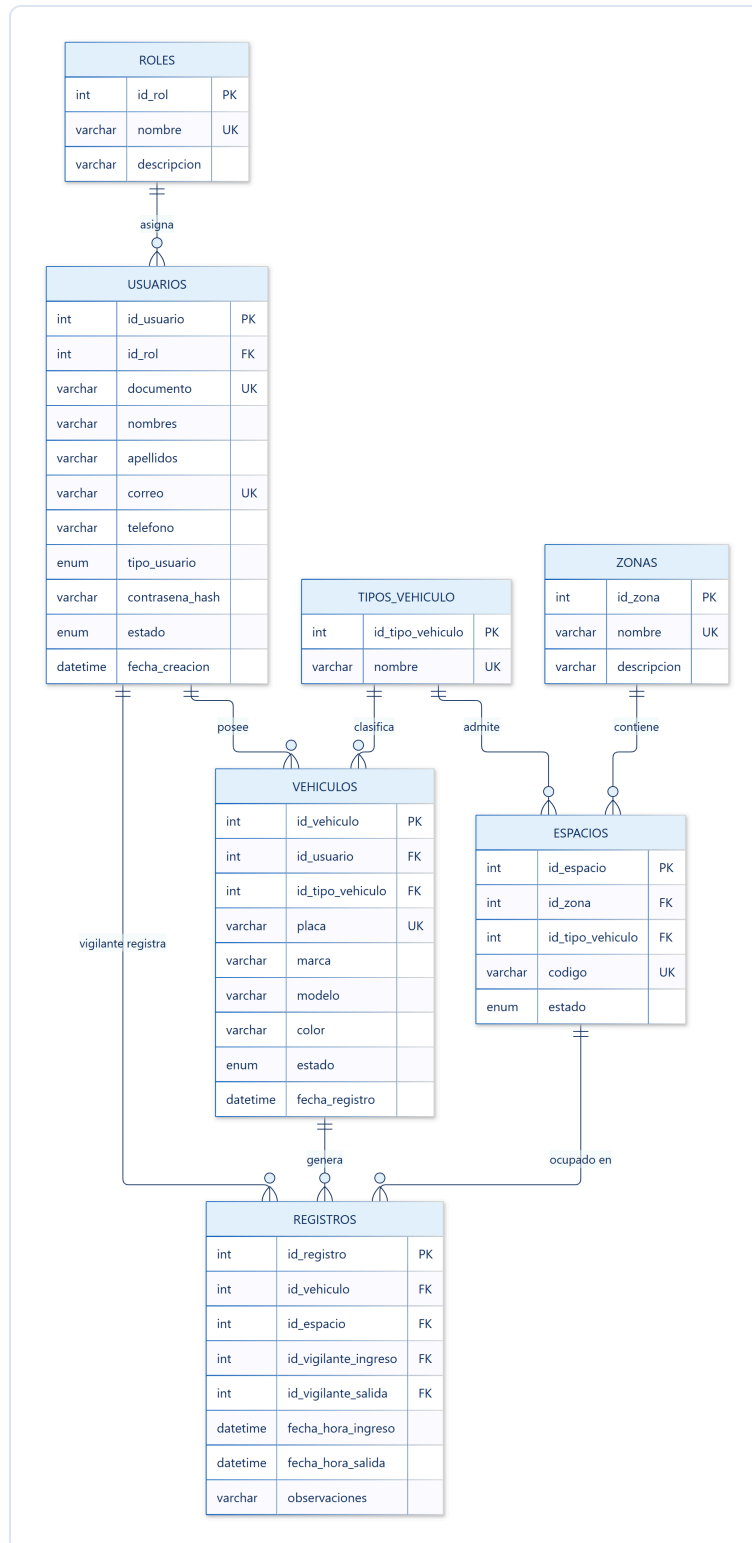


Figura 2. Modelo entidad-relación propuesto del sistema PARQUEADERO.

Notación: PK = llave primaria, FK = llave foránea, UK = valor único. Las líneas usan notación "pata de gallo": || exactamente uno, o{ cero o muchos.

7.2 Entidades y atributos

7.2.1 roles

Roles de acceso al sistema.

Atributo	Tipo de dato	Llave / restricción
id_rol	INT	PK, AUTO_INCREMENT
nombre	VARCHAR(30)	NOT NULL, UNIQUE
descripcion	VARCHAR(150)	NULL

7.2.2 usuarios

Personas registradas: estudiantes, profesores, personal, vigilantes y administradores.

Atributo	Tipo de dato	Llave / restricción
id_usuario	INT	PK, AUTO_INCREMENT
id_rol	INT	FK → roles.id_rol, NOT NULL
documento	VARCHAR(20)	NOT NULL, UNIQUE
nombres	VARCHAR(80)	NOT NULL
apellidos	VARCHAR(80)	NOT NULL
correo	VARCHAR(120)	NOT NULL, UNIQUE
telefono	VARCHAR(20)	NULL
tipo_usuario	ENUM('ESTUDIANTE', 'PROFESOR', 'PERSONAL')	NOT NULL
contrasena_hash	VARCHAR(255)	NOT NULL
estado	ENUM('ACTIVO', 'INACTIVO')	NOT NULL, DEFAULT 'ACTIVO'
fecha_creacion	DATETIME	NOT NULL, DEFAULT CURRENT_TIMESTAMP

7.2.3 tipos_vehiculo

Catálogo de tipos de vehículo (carro, moto, bicicleta).

Atributo	Tipo de dato	Llave / restricción
id_tipo_vehiculo	INT	PK, AUTO_INCREMENT
nombre	VARCHAR(30)	NOT NULL, UNIQUE

7.2.4 vehiculos

Vehículos autorizados, cada uno asociado a un usuario.

Atributo	Tipo de dato	Llave / restricción
id_vehiculo	INT	PK, AUTO_INCREMENT
id_usuario	INT	FK → usuarios.id_usuario, NOT NULL
id_tipo_vehiculo	INT	FK → tipos_vehiculo.id_tipo_vehiculo, NOT NULL
placa	VARCHAR(10)	NOT NULL, UNIQUE
marca	VARCHAR(40)	NULL
modelo	VARCHAR(40)	NULL
color	VARCHAR(30)	NULL
estado	ENUM('ACTIVO' , 'INACTIVO')	NOT NULL, DEFAULT 'ACTIVO'
fecha_registro	DATETIME	NOT NULL, DEFAULT CURRENT_TIMESTAMP

7.2.5 zonas

Áreas en que se divide el parqueadero.

Atributo	Tipo de dato	Llave / restricción
id_zona	INT	PK, AUTO_INCREMENT
nombre	VARCHAR(50)	NOT NULL, UNIQUE
descripcion	VARCHAR(150)	NULL

7.2.6 espacios

Espacios individuales de estacionamiento.

Atributo	Tipo de dato	Llave / restricción
id_espacio	INT	PK, AUTO_INCREMENT
id_zona	INT	FK → zonas.id_zona, NOT NULL
id_tipo_vehiculo	INT	FK → tipos_vehiculo.id_tipo_vehiculo, NOT NULL
codigo	VARCHAR(10)	NOT NULL, UNIQUE
estado	ENUM('DISPONIBLE', 'OCUPADO', 'INHABILITADO')	NOT NULL, DEFAULT 'DISPONIBLE'

7.2.7 registros

Movimientos de ingreso y salida de vehículos.

Atributo	Tipo de dato	Llave / restricción
id_registro	INT	PK, AUTO_INCREMENT
id_vehiculo	INT	FK → vehiculos.id_vehiculo, NOT NULL
id_espacio	INT	FK → espacios.id_espacio, NOT NULL
id_vigilante_ingreso	INT	FK → usuarios.id_usuario, NOT NULL
id_vigilante_salida	INT	FK → usuarios.id_usuario, NULL
fecha_hora_ingreso	DATETIME	NOT NULL, DEFAULT CURRENT_TIMESTAMP
fecha_hora_salida	DATETIME	NULL (NULL = vehículo dentro del parqueadero)
observaciones	VARCHAR(255)	NULL

7.3 Relaciones y cardinalidades

Relación	Entidades	Cardinalidad	Llave foránea	Descripción
asigna	roles — usuarios	1 : N	usuarios.id_rol	Un rol puede asignarse a muchos usuarios; cada usuario tiene un solo rol.

Relación	Entidades	Cardinalidad	Llave foránea	Descripción
posee	usuarios — vehiculos	1 : N	vehiculos.id_usuario	Un usuario puede tener varios vehículos; cada vehículo pertenece a un usuario.
clasifica	tipos_vehiculo — vehiculos	1 : N	vehiculos.id_tipo_vehiculo	Cada vehículo es de un tipo (carro, moto, bicicleta).
admite	tipos_vehiculo — espacios	1 : N	espacios.id_tipo_vehiculo	Cada espacio está destinado a un tipo de vehículo.
contiene	zonas — espacios	1 : N	espacios.id_zona	Una zona agrupa muchos espacios.
genera	vehiculos — registros	1 : N	registros.id_vehiculo	Un vehículo acumula muchos registros de ingreso/salida.
ocupado en	espacios — registros	1 : N	registros.id_espacio	Un espacio aparece en muchos registros a lo largo del tiempo.
vigilante registra	usuarios — registros	1 : N (x2)	registros.id_vigilante_ingreso, registros.id_vigilante_salida	Identifica al vigilante que registró el ingreso y, cuando aplica, la salida.

7.4 Reglas de negocio reflejadas en el modelo

- Un registro con `fecha_hora_salida` en NULL indica que el vehículo está dentro del parqueadero.
- Un vehículo no puede tener más de un registro abierto al mismo tiempo (validación en el backend).
- Solo se asignan espacios con estado `DISPONIBLE` y del mismo tipo que el vehículo.

8. Base de datos MySQL

Estado de la base de datos

El repositorio **no contiene todavía scripts SQL ni consultas** en el código. El siguiente script es la implementación en MySQL del modelo entidad-relación propuesto (documento 05). Los signos `?` representan parámetros que el backend debe enviar mediante consultas preparadas, para prevenir inyección SQL.

8.1 Creación de la base de datos y tablas

```
-- =====
-- PARQUEADERO - Systems & Programación
-- Script MySQL 8 (DISEÑO PROPUESTO)
-- =====
CREATE DATABASE IF NOT EXISTS parqueadero_db
  CHARACTER SET utf8mb4 COLLATE utf8mb4_unicode_ci;
USE parqueadero_db;

CREATE TABLE roles (
  id_rol      INT AUTO_INCREMENT PRIMARY KEY,
  nombre      VARCHAR(30) NOT NULL UNIQUE,
  descripcion VARCHAR(150) NULL
) ENGINE=InnoDB;

CREATE TABLE usuarios (
  id_usuario  INT AUTO_INCREMENT PRIMARY KEY,
  id_rol      INT NOT NULL,
  documento   VARCHAR(20) NOT NULL UNIQUE,
  nombres     VARCHAR(80) NOT NULL,
  apellidos   VARCHAR(80) NOT NULL,
  correo      VARCHAR(120) NOT NULL UNIQUE,
  telefono    VARCHAR(20) NULL,
  tipo_usuario ENUM('ESTUDIANTE', 'PROFESOR', 'PERSONAL') NOT NULL,
  contrasena_hash VARCHAR(255) NOT NULL,
  estado      ENUM('ACTIVO', 'INACTIVO') NOT NULL DEFAULT 'ACTIVO',
  fecha_creacion DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
  CONSTRAINT fk_usuarios_rol FOREIGN KEY (id_rol)
    REFERENCES roles(id_rol) ON UPDATE CASCADE ON DELETE RESTRICT
) ENGINE=InnoDB;

CREATE TABLE tipos_vehiculo (
  id_tipo_vehiculo INT AUTO_INCREMENT PRIMARY KEY,
  nombre          VARCHAR(30) NOT NULL UNIQUE
) ENGINE=InnoDB;

CREATE TABLE vehiculos (
  id_vehiculo  INT AUTO_INCREMENT PRIMARY KEY,
  id_usuario   INT NOT NULL,
  id_tipo_vehiculo INT NOT NULL,
```

```
placa          VARCHAR(10) NOT NULL UNIQUE,
marca          VARCHAR(40) NULL,
modelo        VARCHAR(40) NULL,
color         VARCHAR(30) NULL,
estado        ENUM('ACTIVO','INACTIVO') NOT NULL DEFAULT 'ACTIVO',
fecha_registro DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
CONSTRAINT fk_vehiculos_usuario FOREIGN KEY (id_usuario)
  REFERENCES usuarios(id_usuario) ON UPDATE CASCADE ON DELETE RESTRICT,
CONSTRAINT fk_vehiculos_tipo FOREIGN KEY (id_tipo_vehiculo)
  REFERENCES tipos_vehiculo(id_tipo_vehiculo) ON UPDATE CASCADE ON DELETE RESTRICT
) ENGINE=InnoDB;

CREATE TABLE zonas (
  id_zona      INT AUTO_INCREMENT PRIMARY KEY,
  nombre       VARCHAR(50) NOT NULL UNIQUE,
  descripcion  VARCHAR(150) NULL
) ENGINE=InnoDB;

CREATE TABLE espacios (
  id_espacio   INT AUTO_INCREMENT PRIMARY KEY,
  id_zona      INT NOT NULL,
  id_tipo_vehiculo INT NOT NULL,
  codigo       VARCHAR(10) NOT NULL UNIQUE,
  estado       ENUM('DISPONIBLE','OCUPADO','INHABILITADO') NOT NULL DEFAULT 'DISPONIBLE',
CONSTRAINT fk_espacios_zona FOREIGN KEY (id_zona)
  REFERENCES zonas(id_zona) ON UPDATE CASCADE ON DELETE RESTRICT,
CONSTRAINT fk_espacios_tipo FOREIGN KEY (id_tipo_vehiculo)
  REFERENCES tipos_vehiculo(id_tipo_vehiculo) ON UPDATE CASCADE ON DELETE RESTRICT
) ENGINE=InnoDB;

CREATE TABLE registros (
  id_registro  INT AUTO_INCREMENT PRIMARY KEY,
  id_vehiculo  INT NOT NULL,
  id_espacio   INT NOT NULL,
  id_vigilante_ingreso INT NOT NULL,
  id_vigilante_salida INT NULL,
  fecha_hora_ingreso DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
  fecha_hora_salida DATETIME NULL,
  observaciones VARCHAR(255) NULL,
CONSTRAINT fk_registros_vehiculo FOREIGN KEY (id_vehiculo)
  REFERENCES vehiculos(id_vehiculo) ON UPDATE CASCADE ON DELETE RESTRICT,
CONSTRAINT fk_registros_espacio FOREIGN KEY (id_espacio)
  REFERENCES espacios(id_espacio) ON UPDATE CASCADE ON DELETE RESTRICT,
CONSTRAINT fk_registros_vig_ingreso FOREIGN KEY (id_vigilante_ingreso)
  REFERENCES usuarios(id_usuario) ON UPDATE CASCADE ON DELETE RESTRICT,
CONSTRAINT fk_registros_vig_salida FOREIGN KEY (id_vigilante_salida)
  REFERENCES usuarios(id_usuario) ON UPDATE CASCADE ON DELETE RESTRICT,
CONSTRAINT chk_registros_fechas
  CHECK (fecha_hora_salida IS NULL OR fecha_hora_salida >= fecha_hora_ingreso)
) ENGINE=InnoDB;

CREATE INDEX idx_registros_abiertos ON registros (id_vehiculo, fecha_hora_salida);
CREATE INDEX idx_registros_ingreso ON registros (fecha_hora_ingreso);
```

8.2 Resumen de restricciones

Tipo	Aplicación
PRIMARY KEY	Todas las tablas usan una llave primaria entera AUTO_INCREMENT.
FOREIGN KEY	9 llaves foráneas con ON DELETE RESTRICT para impedir borrar datos referenciados.
UNIQUE	roles.nombre, usuarios.documento, usuarios.correo, tipos_vehiculo.nombre, vehiculos.placa, zonas.nombre, espacios.codigo.
ENUM	tipo_usuario, estados de usuario, vehículo y espacio.
CHECK	La fecha de salida no puede ser anterior a la de ingreso.
DEFAULT	Estados iniciales y fechas de creación automáticas.

8.3 Datos de prueba

Aclaración

El proyecto no tiene datos reales. Los siguientes registros son **ficticios** y solo sirven para probar el sistema.

```
-- -----
-- Datos de prueba (FICTICIOS, solo para pruebas)
-- contrasena_hash: reemplazar por hashes bcrypt generados por el backend
-- -----
INSERT INTO roles (nombre, descripcion) VALUES
  ('ADMINISTRADOR', 'Gestiona usuarios, vehículos, zonas y espacios'),
  ('VIGILANTE', 'Registra ingresos y salidas en portería'),
  ('USUARIO', 'Miembro de la institución con vehículos registrados');

INSERT INTO tipos_vehiculo (nombre) VALUES ('CARRO'), ('MOTO'), ('BICICLETA');

INSERT INTO zonas (nombre, descripcion) VALUES
  ('Zona A', 'Carros - entrada principal'),
  ('Zona B', 'Motos'),
  ('Zona C', 'Bicicletas');

INSERT INTO espacios (id_zona, id_tipo_vehiculo, codigo) VALUES
  (1, 1, 'A-01'), (1, 1, 'A-02'), (1, 1, 'A-03'),
  (2, 2, 'B-01'), (2, 2, 'B-02'),
  (3, 3, 'C-01');

INSERT INTO usuarios (id_rol, documento, nombres, apellidos, correo, tipo_usuario, contrasena_hash)
VALUES
  (1, '100000001', 'Admin', 'Prueba', 'admin@institucion.edu.co', 'PERSONAL', '<hash>'),
```

```
(2, '1000000002', 'Vigilante', 'Prueba', 'vigilante@institucion.edu.co', 'PERSONAL', '<hash>'),
(3, '1000000003', 'Usuario', 'Uno', 'usuario1@institucion.edu.co', 'ESTUDIANTE', '<hash>'),
(3, '1000000004', 'Usuario', 'Dos', 'usuario2@institucion.edu.co', 'PROFESOR', '<hash>');

INSERT INTO vehiculos (id_usuario, id_tipo_vehiculo, placa, marca, modelo, color) VALUES
(3, 2, 'ABC12D', 'Marca X', '2022', 'Negro'),
(4, 1, 'XYZ123', 'Marca Y', '2020', 'Gris');
```

8.4 Consultas principales del sistema

8.4.1 1. Inicio de sesión: obtener usuario por correo (RF-01)

```
SELECT u.id_usuario, u.nombres, u.apellidos, u.contrasena_hash, u.estado, r.nombre AS rol
FROM usuarios u
JOIN roles r ON r.id_rol = u.id_rol
WHERE u.correo = ?;
```

8.4.2 2. Buscar vehículo por placa y validar autorización (RF-09, RF-10)

```
SELECT v.id_vehiculo, v.placa, v.estado, t.id_tipo_vehiculo, t.nombre AS tipo,
       CONCAT(u.nombres, ' ', u.apellidos) AS propietario, u.estado AS estado_usuario,
       (SELECT COUNT(*) FROM registros r
        WHERE r.id_vehiculo = v.id_vehiculo AND r.fecha_hora_salida IS NULL) AS ingresos_abiertos
FROM vehiculos v
JOIN usuarios u ON u.id_usuario = v.id_usuario
JOIN tipos_vehiculo t ON t.id_tipo_vehiculo = v.id_tipo_vehiculo
WHERE v.placa = ?;
```

8.4.3 3. Espacios disponibles para un tipo de vehículo (RF-09)

```
SELECT e.id_espacio, e.codigo, z.nombre AS zona
FROM espacios e
JOIN zonas z ON z.id_zona = e.id_zona
WHERE e.estado = 'DISPONIBLE' AND e.id_tipo_vehiculo = ?
ORDER BY z.nombre, e.codigo;
```

8.4.4 4. Registrar ingreso (transacción) (RF-09)

```
START TRANSACTION;
INSERT INTO registros (id_vehiculo, id_espacio, id_vigilante_ingreso)
VALUES (?, ?, ?);
UPDATE espacios SET estado = 'OCUPADO'
WHERE id_espacio = ? AND estado = 'DISPONIBLE';
COMMIT;
```

8.4.5 5. Registrar salida (transacción) (RF-11)

```
START TRANSACTION;
UPDATE registros
SET fecha_hora_salida = NOW(), id_vigilante_salida = ?
WHERE id_registro = ? AND fecha_hora_salida IS NULL;
UPDATE espacios SET estado = 'DISPONIBLE'
WHERE id_espacio = ?;
COMMIT;
```

8.4.6 6. Disponibilidad por zona y tipo (RF-12)

```
SELECT z.nombre AS zona, t.nombre AS tipo,
       SUM(e.estado = 'DISPONIBLE') AS disponibles,
       SUM(e.estado = 'OCUPADO') AS ocupados,
       SUM(e.estado = 'INHABILITADO') AS inhabilitados,
       COUNT(*) AS total
FROM espacios e
JOIN zonas z      ON z.id_zona = e.id_zona
JOIN tipos_vehiculo t ON t.id_tipo_vehiculo = e.id_tipo_vehiculo
GROUP BY z.nombre, t.nombre
ORDER BY z.nombre;
```

8.4.7 7. Vehículos dentro del parqueadero (RF-13)

```
SELECT r.id_registro, v.placa, CONCAT(u.nombres, ' ', u.apellidos) AS propietario,
       e.codigo AS espacio, r.fecha_hora_ingreso
FROM registros r
JOIN vehiculos v ON v.id_vehiculo = r.id_vehiculo
JOIN usuarios u  ON u.id_usuario = v.id_usuario
JOIN espacios e  ON e.id_espacio = r.id_espacio
WHERE r.fecha_hora_salida IS NULL
ORDER BY r.fecha_hora_ingreso;
```

8.4.8 8. Historial por placa y rango de fechas (RF-14)

```
SELECT v.placa, e.codigo AS espacio, r.fecha_hora_ingreso, r.fecha_hora_salida,
       TIMEDIFF(MINUTE, r.fecha_hora_ingreso, r.fecha_hora_salida) AS minutos
FROM registros r
JOIN vehiculos v ON v.id_vehiculo = r.id_vehiculo
JOIN espacios e  ON e.id_espacio = r.id_espacio
WHERE v.placa = ? AND r.fecha_hora_ingreso BETWEEN ? AND ?
ORDER BY r.fecha_hora_ingreso DESC;
```

8.4.9 9. Vehículos de un usuario (RF-16)

```
SELECT v.placa, t.nombre AS tipo, v.marca, v.modelo, v.color, v.estado
FROM vehiculos v
```

```
JOIN tipos_vehiculo t ON t.id_tipo_vehiculo = v.id_tipo_vehiculo  
WHERE v.id_usuario = ?;
```

8.4.10 10. Reporte de ingresos por día (RF-15)

```
SELECT DATE(r.fecha_hora_ingreso) AS dia, COUNT(*) AS ingresos  
FROM registros r  
WHERE r.fecha_hora_ingreso BETWEEN ? AND ?  
GROUP BY DATE(r.fecha_hora_ingreso)  
ORDER BY dia;
```

9. Funcionamiento del sistema

Funcionamiento propuesto

A la fecha, la aplicación desplegada muestra solo la página de ejemplo de la plantilla Vite + React. El siguiente flujo describe el **funcionamiento propuesto** del sistema una vez implementado, de acuerdo con los requisitos funcionales e historias de usuario.

9.1 1. Acceso a la aplicación

1. La persona abre la dirección web del sistema **[URL de despliegue pendiente]** desde el navegador del computador o del teléfono.
2. Al ser una PWA, el navegador ofrece la opción **"Instalar aplicación"**, que agrega el ícono de PARQUEADERO al escritorio o a la pantalla de inicio.

9.2 2. Inicio de sesión

1. El sistema muestra el formulario de inicio de sesión (correo y contraseña).
2. El backend valida las credenciales. Si son correctas, identifica el rol y muestra el panel correspondiente.
3. Las cuentas son creadas por el administrador; no hay registro libre de usuarios.

9.3 3. Configuración inicial (Administrador)

1. Crea las zonas del parqueadero (por ejemplo, Zona A para carros, Zona B para motos).
2. Crea los espacios de cada zona indicando código y tipo de vehículo.
3. Registra a los usuarios de la institución y sus vehículos.

9.4 4. Operación diaria (Vigilante)

1. **Ingreso:** cuando llega un vehículo, el vigilante digita la placa. El sistema verifica que esté autorizado, muestra el propietario y los espacios disponibles; el vigilante selecciona el espacio y confirma.
2. **Disponibilidad:** el panel muestra en todo momento los espacios libres y ocupados por zona.
3. **Salida:** cuando el vehículo sale, el vigilante busca la placa en la lista de vehículos dentro del parqueadero y registra la salida; el espacio queda libre.

9.5 5. Consultas (Usuario)

1. Consulta cuántos espacios hay disponibles para su tipo de vehículo.
2. Consulta sus vehículos registrados y el historial de ingresos y salidas.

9.6 6. Reportes (Administrador)

Consulta el historial completo y el reporte de ingresos por día y ocupación por zona en un rango de fechas.

9.7 7. Cierre de sesión

Cualquier usuario puede cerrar su sesión desde el menú; el sistema regresa a la pantalla de inicio de sesión.

9.8 Diagrama de flujo: ingreso de un vehículo

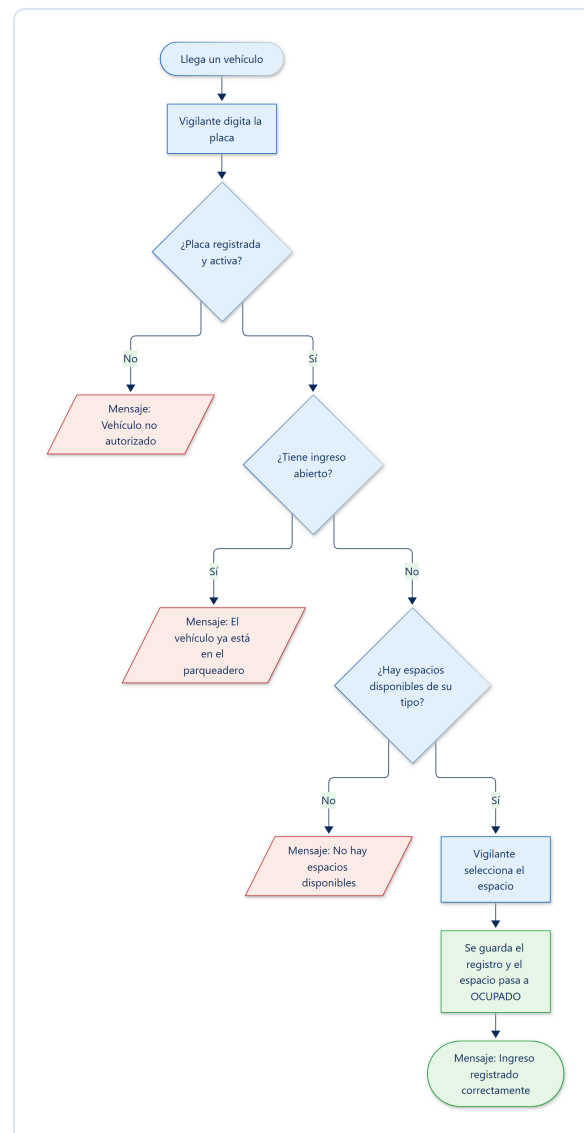


Figura 3. Flujo propuesto para el registro de ingreso de un vehículo.

10. Tecnologías y documentación técnica

10.1 Estructura de carpetas EXISTENTE

```

Parkinloock-/
├── .gitignore           Archivos excluidos de Git (node_modules, .env, dist, ...)
├── LICENSE             Licencia Apache 2.0
├── README.md           Descripción del proyecto
├── backend-Parkinlock/ Backend (vacío actualmente)
└── frontend-Parkinlock-/ Frontend PWA
    ├── index.html      Página HTML principal (íconos, título "Parkinlock")
    ├── package.json     Dependencias y scripts
    ├── vite.config.js   Configuración de Vite, React y PWA (manifest)
    ├── pwa-assets.config.js Configuración del generador de íconos
    ├── .oxlinttrc.json  Configuración del linter
    ├── public/          Archivos estáticos e íconos PWA
    │   ├── favicon.svg   Imagen base de los íconos
    │   ├── favicon.ico
    │   ├── pwa-64x64.png, pwa-192x192.png, pwa-512x512.png
    │   ├── maskable-icon-512x512.png
    │   └── apple-touch-icon-180x180.png
    └── src/
        ├── main.jsx      Punto de entrada: monta <App /> en #root
        ├── App.jsx       Componente principal (plantilla de ejemplo)
        ├── App.css, index.css Estilos
        └── assets/       Imágenes
    
```

10.2 Archivos principales

Archivo	Función
frontend-Parkinlock-/vite.config.js	Registra los plugins de React y PWA. Define el manifest (nombre "Parkinlock", colores, modo standalone e íconos) y la actualización automática del service worker.
frontend-Parkinlock-/pwa-assets.config.js	Usa el preset <i>minimal-2023</i> para generar los íconos desde public/favicon.svg.
frontend-Parkinlock-/src/main.jsx	Inicializa React en modo estricto y renderiza el componente App.
frontend-Parkinlock-/package.json	Lista dependencias y scripts de ejecución.
frontend-Parkinlock-/index.html	Documento HTML con enlaces a íconos, color de tema y contenedor #root.

10.3 Tecnologías

Capa	Tecnología	Estado
Frontend	React 19.2, Vite 8.3, JavaScript (JSX)	EXISTENTE
PWA	vite-plugin-pwa 1.3 (Workbox), @vite-pwa/assets-generator 1.0.4	EXISTENTE
Calidad de código	oxlint 1.81	EXISTENTE
Control de versiones	Git + GitHub	EXISTENTE
Entorno de ejecución	Node.js + npm	EXISTENTE
Backend	[Pendiente de definir]	PROPUESTO
Base de datos	MySQL (Clever Cloud)	PROPUESTO
Despliegue frontend	Vercel	PROPUESTO
Editor	Cursor	Usado en el desarrollo

10.4 Funcionamiento técnico

Vite sirve la aplicación React durante el desarrollo con recarga instantánea. En producción, `npm run build` genera la carpeta `dist/` con los archivos optimizados; vite-plugin-pwa agrega `manifest.webmanifest`, `registerSW.js` y `sw.js` (Workbox), que almacena en caché los recursos para cargas más rápidas e instalación como aplicación.

10.5 Conexión con la base de datos PROPUESTO

El frontend **nunca** se conecta directamente a MySQL. El backend se conectará usando variables de entorno definidas en un archivo `.env` (excluido de Git):

```
DB_HOST=      # host entregado por Clever Cloud
DB_PORT=3306
DB_USER=
DB_PASSWORD=
DB_NAME=parqueadero_db
```

En el frontend, la URL de la API se configurará con una variable de Vite, por ejemplo `VITE_API_URL` [Pendiente de completar].

10.6 Ejecución local

Requisitos: Node.js (versión LTS) y Git.

```
git clone https://github.com/camilotobon34-sudo/Parkinloock-.git
cd Parkinloock-/frontend-Parkinlock-
npm install
npm run dev          # abre http://localhost:5173
```

Script	Comando	Descripción
dev	<code>npm run dev</code>	Servidor de desarrollo.
build	<code>npm run build</code>	Compila para producción en dist/ (incluye service worker).
preview	<code>npm run preview</code>	Sirve localmente el build de producción.
lint	<code>npm run lint</code>	Analiza el código con oxlint.
generate-pwa-assets	<code>npm run generate-pwa-assets</code>	Regenera los íconos PWA desde public/favicon.svg.

Importante

Los comandos `npm` del frontend deben ejecutarse **dentro** de la carpeta `frontend-Parkinlock-`. Si se ejecutan en la raíz del repositorio, se crea un `package.json` incorrecto.

10.7 Configuración

- **Nombre y colores de la PWA:** en `vite.config.js`, sección `manifest` (actualmente color `#ffffff`; se recomienda usar el azul del logo).
- **Íconos:** reemplazar `public/favicon.svg` por el logo oficial y ejecutar `npm run generate-pwa-assets`.
- **Título de la página:** etiqueta `<title>` en `index.html`.

10.8 Despliegue

Sin despliegue registrado

No se encontró evidencia de despliegue en el repositorio. Siguiendo la metodología del curso, el procedimiento propuesto es: (1) importar el repositorio en Vercel, (2) seleccionar `frontend-Parkinlock-` como directorio raíz, (3) usar el comando de build `npm run build` y el directorio de salida `dist`. URL de producción: **[Pendiente de completar]**

11. Estado del proyecto

11.1 Estado actual del proyecto

Componente	Estado	Detalle
Repositorio GitHub	EXISTENTE	Carpetas de frontend y backend, README, licencia Apache 2.0.
Frontend PWA	EXISTENTE	Plantilla React + Vite con manifest, service worker e íconos generados.
Pantallas del parqueadero	PROPUESTO	Por desarrollar.
Backend / API	PROPUESTO	Carpeta vacía; tecnología por definir.
Base de datos MySQL	PROPUESTO	Modelo y script diseñados en los documentos 05 y 06.
Despliegue	PENDIENTE	Sin evidencia de despliegue.

11.2 Diferencias entre el video de referencia y el proyecto

Aspecto	Video de referencia	Proyecto actual
Contenido	Sesión de clase sobre metodología de desarrollo de otro proyecto.	Sistema de parqueadero institucional.
Creación del frontend	Plantilla PWA con Vite.	Plantilla Vite + React con vite-plugin-pwa (equivalente).
Base de datos	MySQL en Clever Cloud.	No implementada; propuesta MySQL.
Despliegue	Vercel.	No realizado.
Funcionalidades de parqueadero	No se muestran.	No implementadas; diseño propuesto en esta documentación.

12. Conclusiones

- El proyecto PARQUEADERO cuenta con una base técnica sólida: repositorio organizado en GitHub y un frontend configurado como aplicación web progresiva con React, Vite y vite-plugin-pwa, instalable y con caché mediante service worker.
- Las funcionalidades del sistema de parqueadero aún no están implementadas; esta documentación define un diseño completo (16 requisitos funcionales, 14 no funcionales, 12 historias de usuario y un modelo de datos de 7 tablas) que sirve como guía para el desarrollo.
- El modelo de datos propuesto permite controlar ingresos y salidas, conocer la disponibilidad por zona y tipo de vehículo y conservar el historial, cubriendo la problemática descrita en el repositorio.
- Los siguientes pasos son: definir la tecnología del backend, crear la base de datos MySQL con el script del documento 06, desarrollar las pantallas del frontend y desplegar la aplicación.
- Esta documentación debe actualizarse a medida que se implementen las funcionalidades, reemplazando las secciones marcadas como propuestas o pendientes por la información real.